

Feedback first: thresholds are ML tuning

Jev returns probabilities and our code routes on thresholds, so **tuning those thresholds is ML tuning**, even with no training.

- Our thresholds were set by judgment, never tuned on this data
- Tuning on the rows we report on → optimistic. We need a held-out set.
- Tied to one model build and one question wording. Change either → re-validate.
- Each one trades more auto-routing against more confident mistakes

Next: no threshold changes before the volume run · tune/hold-out split · the business's cost of a wrong signal

Procedure, not firing arrows and then painting the target around them.

10-K numbers in, investment signal out

Jev decides. Arithmetic checks. Code routes.

A generative model only handles what those can't settle.

Code, data, and every run: github.com/skiingfalcon/investment-signal-classifier

Why Jev? Why two levels?

Why not a chat model? It writes text: slow, billed per token, and the confidence it types isn't a probability.

Jev reads once and returns real probabilities that code can route on.

Why not just arithmetic? Here it's the answer key. Real filings also have text (MD&A, footnotes) that arithmetic can't read, so we need to know how far Jev can be trusted.

Why both? Jev alone grades itself. Arithmetic alone can't judge.

Auto only when **both agree**.

What we send Jev

State: one company-year
current + prior facts, `change_pct`, ratios

Questions: 8 typed questions, not a prompt

One pass. No text generated.

A probability for every allowed answer.

The investment choices

Question	Type	Options
revenue_growth	Choice	growing · flat · declining
profitability_trend	Choice	improving · stable · deteriorating · loss_making
ocf_covers_net_income	Noul	yes / no
eps_increased	Noul	yes / no
leverage	Score	0 low · 1 moderate · 2 high
liquidity	Choice	strong · adequate · thin
share_count	Choice	buyback · stable · dilution
overall_signal	Choice	bullish · neutral · bearish

Choice = pick one · Score = place on a scale · Noul = P(yes), 0.5 = cannot tell

One company in → Jev out: AAPL FY2025

	Jev	Rules
revenue_growth	growing · 1.00	growing
profitability_trend	improving · 1.00	improving
ocf_covers_net_income	P(yes) 0.03	no
eps_increased	P(yes) 0.99	yes
leverage	1 · 0.99	1
liquidity	adequate · 1.00	adequate
share_count	buyback · 1.00	buyback
overall_signal	bullish · 1.00	bullish

Confident **and** agrees on all 8 → route = auto

Why not just trust the confidence?

Confidence is the model **grading itself**.

STWD FY2021: Jev said **bullish at 0.99**. The truth was neutral.

So we check it against something that isn't a model: **arithmetic**.

Doubt moves a question to the next stage.

Moving between stages

Stage	Moves on when...	Why the next stage	Cost
1 Jev + rules	not confident, or disagrees	either side could be wrong	1 call
2 Re-ask Jev, else take the rules' answer	still no answer	only now is paying worth it	+1 call
3 Frontier arbiter	bad output or 3-way split	stop guessing	\$ per call
4 Human review	—	—	a person

Each stage only sees what the cheaper one couldn't settle.

Yes/no questions skip the re-ask and go straight to the rules' answer.

What if a tier is missing?

Without...	What happens	In our runs
Rules check	confident mistakes go straight through	qwen: 108 of 427 clean answers wrong, vs 0
Stage 2	every dispute goes to the paid model	qwen: 306 more rows billed
Arbiter	disputes wait for a person	qwen hit this: 22 rows to review
Human review	3-way splits get guessed	a wrong signal ships silently

The tiers are insurance. **You don't know in advance which model you have.**

Results: 336 company-years

	qwen3.8-27b	TypeSafe Jev
Routes	8 auto · 306 verified · 22 review	265 auto · 3 verified · 68 escalated
<code>overall_signal</code> final	98.4%	97.7%
Cost	\$0	\$0.032
Latency p50	15.3 s	249 ms

Same final accuracy. **Very different paths to get there.**

Where it breaks

The stages check **consistency**, not **truth**.

- **Stage 2's re-ask** kept Jev's answer 5 times, and **all 5 were wrong** (STWD included). The rules fallback did the real work.
- **Bad extraction:** Jev and the rules read the same wrong number and agree. Only 2.8% (qwen) / 11.3% (TypeSafe) were caught.
- `share_count` **83.6%** on both backends: a data issue, not a model issue.

Two ways to run Jev

	qwen3.8-27b (local)	TypeSafe Jev (hosted)
Runs on	llama.cpp on the Spark	OpenRouter · <code>typesafe/jev-1.13</code>
Probabilities	we read next-token label logits	Jev returns them natively
Calls per row	8 (one per question)	1 (all 8)
Cost	\$0 marginal	billed per token

Same facts, questions, rules, and policy. **Only the backend changes.**

Takeaway

Jev **decides**. Arithmetic **checks**. Code **routes**.

Cost only rises when there's doubt.

The thresholds that define "doubt" are next, **done properly**.